

Neural Networks Architectures

HW/SW Codesign - Week 7

Today's Topics

- Neural networks review
- Hyperparameter tuning
- Introduction to computer vision
- 2D-Convolution
 - The 2D convolution filter
 - 2D-convolution layer
 - Max pooling
 - Size, memory and compute demands
- Convolutional architectures
 - The ImageNet challenge
 - GoogLeNet: Global average pooling
 - ResNet: Residual connections
 - Xception: Depthwise separable convolution
 - MobileNet
- Sequence modelling
 - Recurrent neural networks (RNN)
 - 1D-Convolution
 - Transformers

Hyperparameter Tuning

How to automate trial and error

Hyperparameters

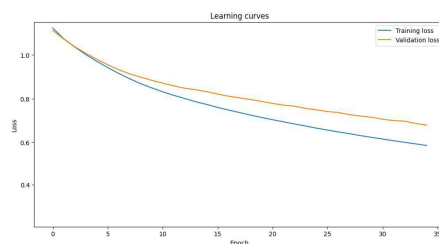
- Model hyperparameters:
 - Input size
 - Number of hidden layers
 - Number of units per layer (for every layer)
 - Batch normalization (True/False for every layer)
 - Activation functions (Linear/Sigmoid/Softmax/ReLU/SiLU etc for every layer)
 - More to come...
- Training hyperparameters:
 - Learning rate
 - Batch size
 - Regularization strength
 - Dropout rate (for every layer)
- Preprocessing and data augmentation hyperparameters

Manual Hyperparameter Tuning

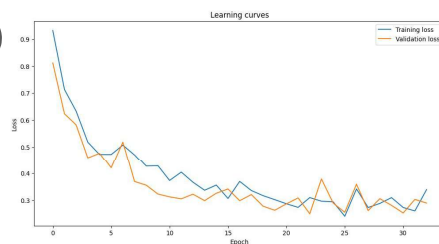
- If underfitting (high training error):
 - Increase number of hidden layers
 - Increase number of neurons per layer
- If overfitting (low training error, high validation error):
 - Decrease number of neurons per layer
 - Decrease batch size
 - Increase regularization strength and dropout rate

Manual Hyperparameter Tuning

- If slow convergence (flat learning curve):
 - Increase learning rate



- If unstable learning (jagged learning curve)
 - Decrease learning rate
 - Increase batch size
 - Decrease dropout rate



Automatic Hyperparameter Tuning

- For each hyperparameter, determine set of values to explore, e.g.:
 - Learning rate: [0.005, 0.001, 0.0005]
 - Batch size: [32, 64, 128]
 - Dropout rate: [0.0, 0.1, 0.2]
 - Number of layers: [2, 3, 4]
 - Layer size: [16, 32, 64]
- Define an **evaluation function**, which trains the model on your data with given hyperparameter values and returns the validation loss or accuracy.
- An iterative search algorithm selects candidate combinations, runs the evaluation function for each combination, and remembers the best combination.

Hyperparameter Tuning: Search Algorithms

- Grid search
 - Evaluate every combination
 - Extremely slow
- Random search
- Bayesian optimization
 - Builds a probabilistic model of model performance over hyperparameters
 - Each iteration:
 - Selects a promising hyperparameter combination
 - Trains/evaluates the model with those hyperparameters
 - Updates the probabilistic model using the observed result

Introduction to Computer Vision

What computer vision models can do

Computer Vision Tasks

- Image classification
- Object detection
- Semantic segmentation

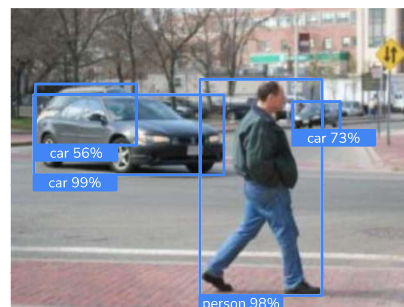
Image Classification

- Predicts a class (binary or multi-class)
- Example applications:
 - Person detection
 - Face recognition
 - Species classification



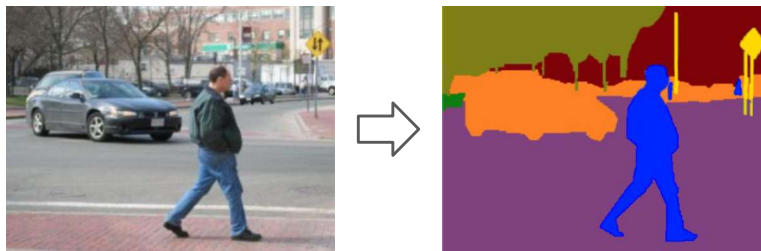
Object Detection

- Detects, classifies and locates multiple objects
- Example applications:
 - Autonomous driving, e.g. pedestrian detection
 - Medical imaging, e.g. tumor detection
 - Precision farming, e.g. weed detection
 - Sports tracking, e.g. football tracking (Veo)
 - Industrial quality control
- Output shape:
 - Large list of anchors, each with:
 - Objectness probability (1)
 - Bounding box coordinates (4)
 - Class probabilities (number of classes)



Semantic Segmentation

- Classifies every pixel of the input image
- Output shape: $input\ height \times input\ width \times number\ of\ classes$
- Example applications:
 - Background removal
 - Autonomous driving



2D-Convolution

How to process images

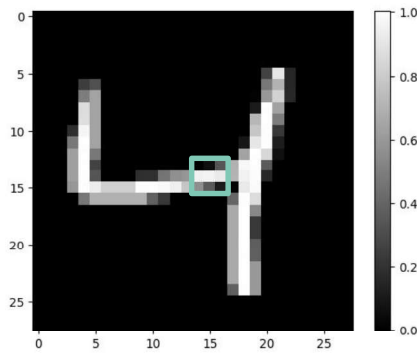
The 2D Convolution Filter

- **Filter** or **kernel**: Matrix of numbers for extracting features
- **Convolution** computes the sum of pointwise multiplications
- Slides over all positions in the input image
- Produces a **feature map**

The 2D Convolution Filter

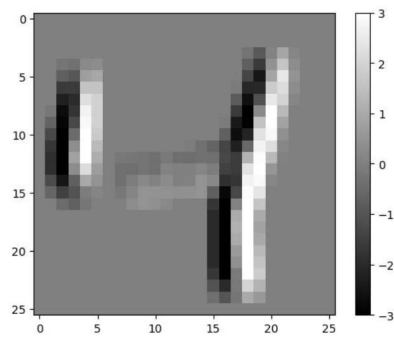
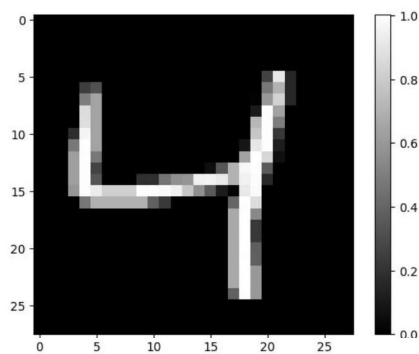
- Properties:
 - Filter height and width
 - Activation: None, ReLU, ...
 - Padding:
 - **Valid**: No padding $\Rightarrow$ Feature map becomes smaller than the input image
 - **Same**: Zero padding $\Rightarrow$ Feature map becomes same size as the input image (stride = 1)
 - Strides (vertical, horizontal)

Vertical Edge Filter



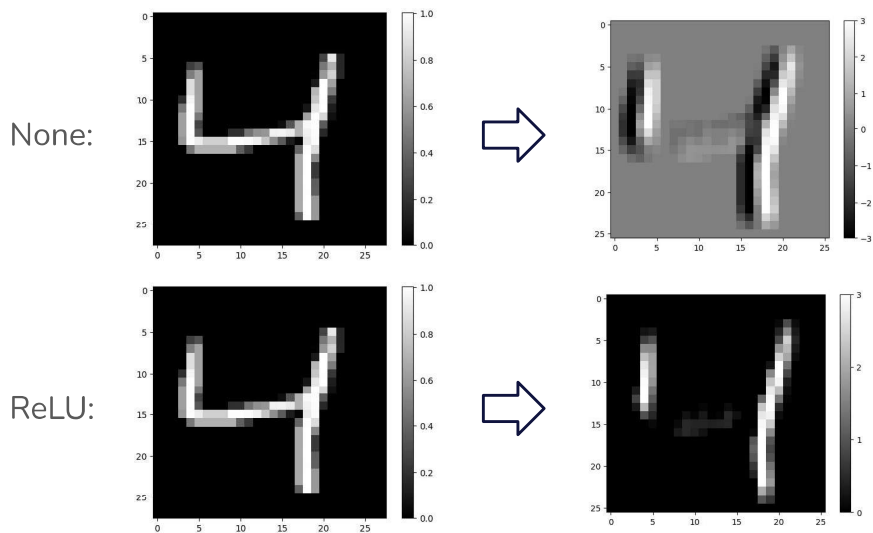
$$\begin{pmatrix} 0.0 & 0.1 & 0.3 \\ 0.9 & 1.0 & 0.9 \\ 0.6 & 0.4 & 0.1 \end{pmatrix} * \begin{pmatrix} 1.0 & 0.0 & -1.0 \\ 1.0 & 0.0 & -1.0 \\ 1.0 & 0.0 & -1.0 \end{pmatrix} = 0.2$$

Vertical Edge Filter

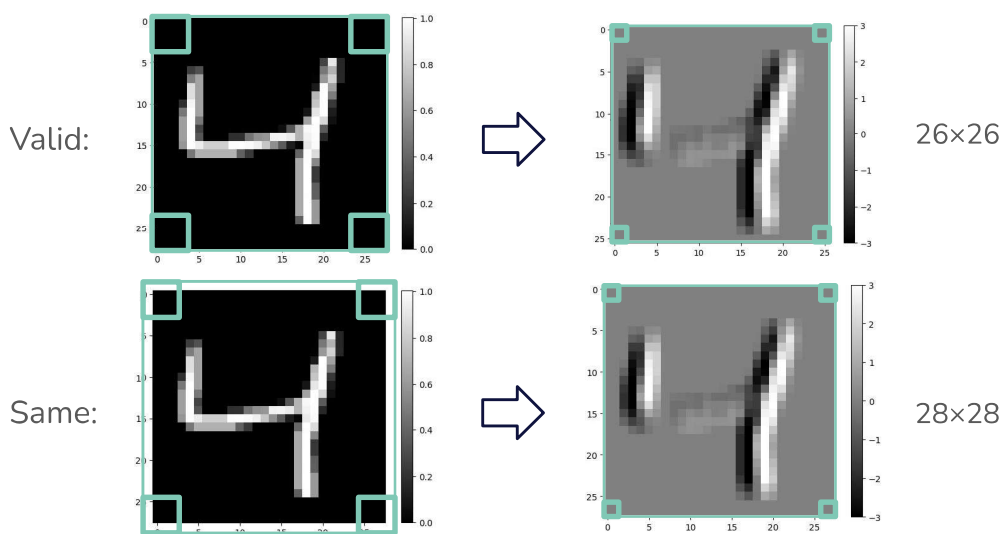


$$* \begin{pmatrix} 1.0 & 0.0 & -1.0 \\ 1.0 & 0.0 & -1.0 \\ 1.0 & 0.0 & -1.0 \end{pmatrix} =$$

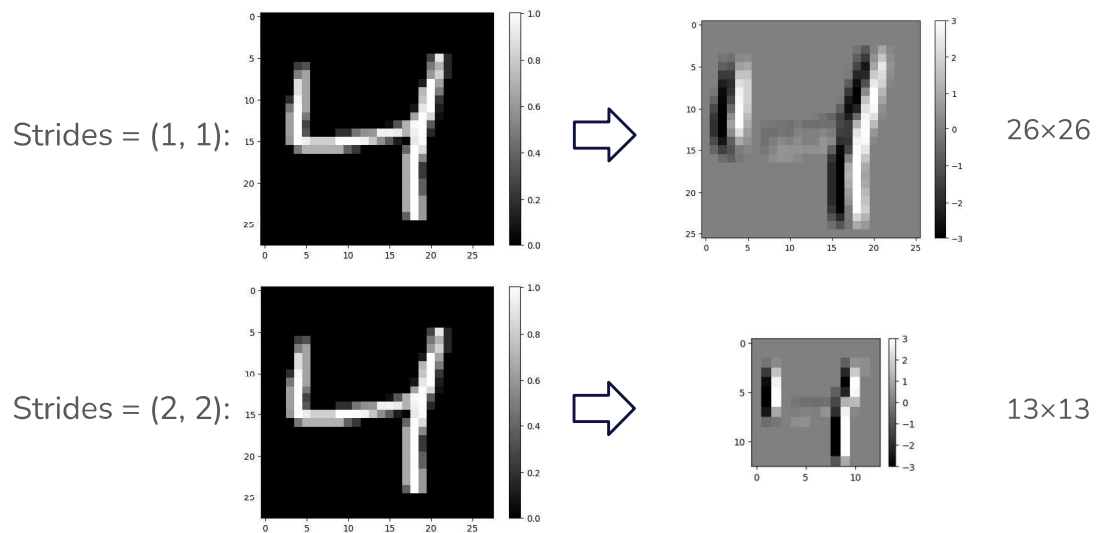
Filter Activation: None vs ReLU



Padding: Valid vs Same



Strides



Multi-Channel Convolution Filter



$$\begin{pmatrix} 0.0 & 1.0 & 0.0 \\ 1.0 & 1.0 & 1.0 \\ 0.0 & 1.0 & 0.0 \end{pmatrix}
 \begin{pmatrix} -1.0 & 0.0 & -1.0 \\ 0.0 & 0.0 & 0.0 \\ -1.0 & 0.0 & -1.0 \end{pmatrix}
 \begin{pmatrix} -1.0 & 0.0 & -1.0 \\ 0.0 & 0.0 & 0.0 \\ -1.0 & 0.0 & -1.0 \end{pmatrix}$$

2D Convolution Layer

The **2D convolution layer** with valid padding and strides (1, 1):

$$a_{i,j,k} = \phi \left(b_k + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} W_{k,u,v,c} \cdot x_{i+u,j+v,c} \right)$$

$\mathbf{x} \in R^{h \times w \times C}$ is the feature tensor,

$\mathbf{W} \in R^{K \times H \times W \times C}$ are the filter weights,

$\mathbf{b} \in R^K$ are the filter biases,

$H, W, C \in \mathbb{Z}^+$ are the filter dimensions,

$\phi(\cdot)$ is the activation function,

$\mathbf{a} \in R^{(h-H+1) \times (w-W+1) \times K}$ is the activation,

K is the number of filters.

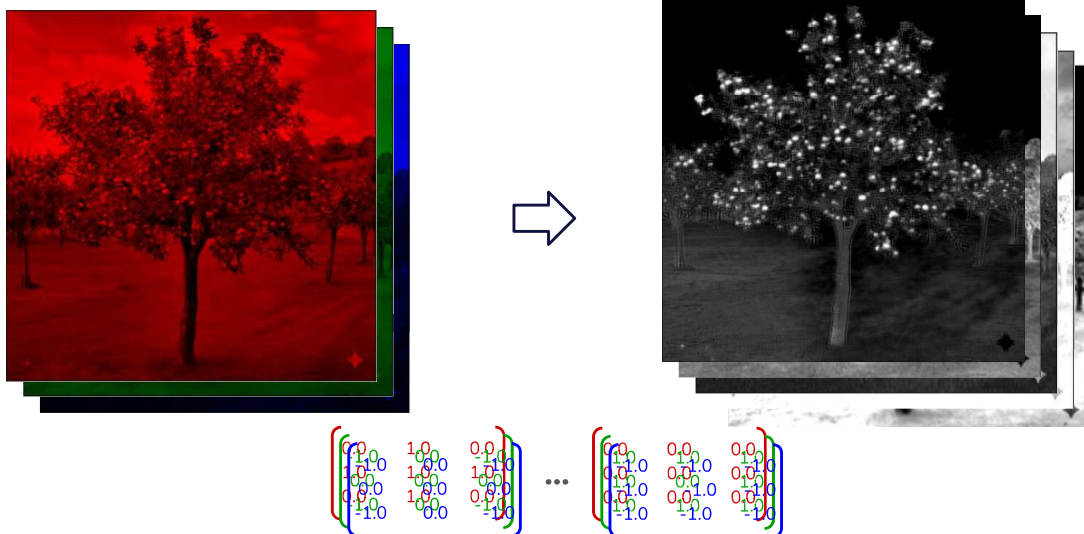
2D Convolution Layer

- Learns convolution filters automatically
- Multiple filters, each with shape (*height* × *width* × *input channels*)
- Each filter produces a new output channel
- Properties:
 - Number of filters, e.g. 32
 - Filter height and width, e.g. 3 × 3
 - *channels* defined by input image
 - Activation
 - Padding: valid or same
 - Strides, by default (1, 1)

2D Convolution Layer

- Layer weights $\mathbf{W}$ are 4D matrices:
 - *number of filters* $\times$ *height* $\times$ *width* $\times$ *number of input channels*
- Layer biases $\mathbf{b}$ are vectors (1D):
 - *number of filters*
- Inputs $\mathbf{x}$ and activations $\mathbf{a}$ are 3D matrices:
 - *height* $\times$ *width* $\times$ *number of channels*

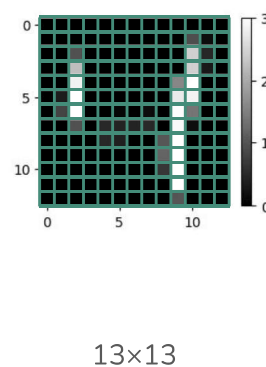
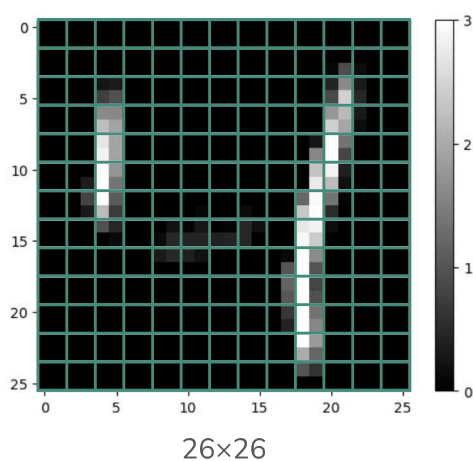
2D Convolution Layer



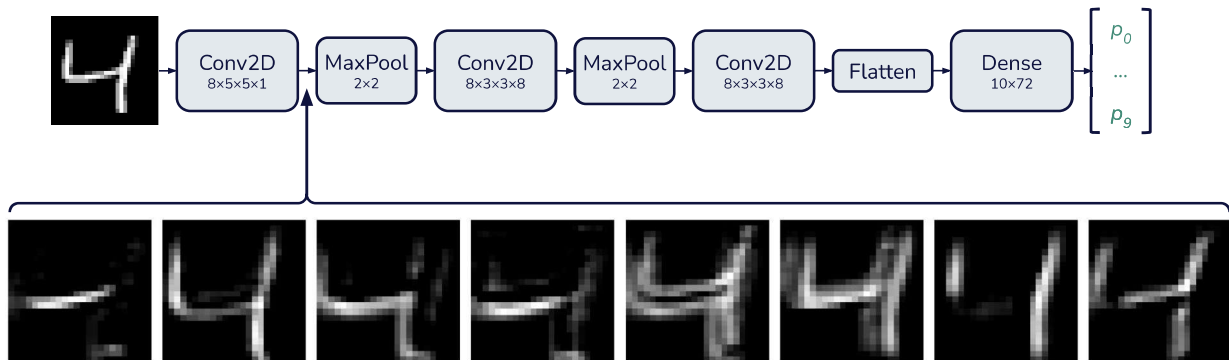
Max Pooling Layer

- Subsamples over width and height
- Inserted between convolution layers
- Purpose:
 - Reduce compute and memory
 - Invariance to translation
- No trainable parameters
- Properties:
 - Pool height and width
 - Padding: valid or same
 - Strides (default: same as height and width)

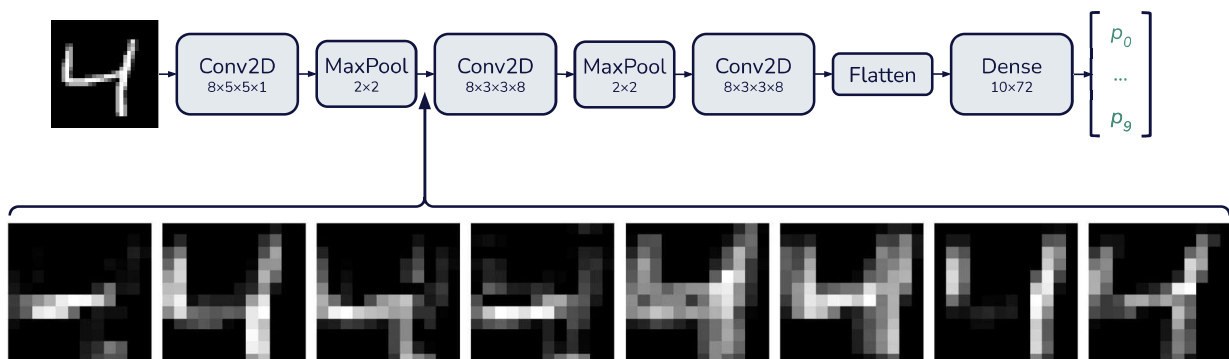
Max Pooling



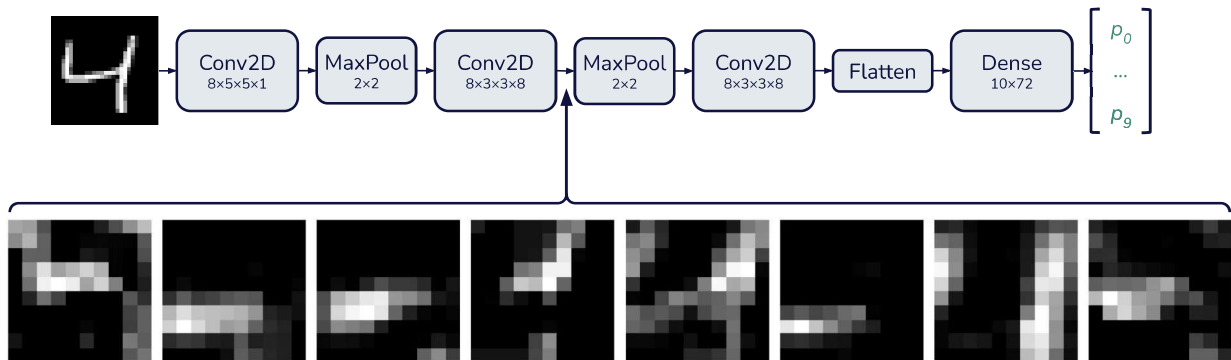
Convolutional Neural Network



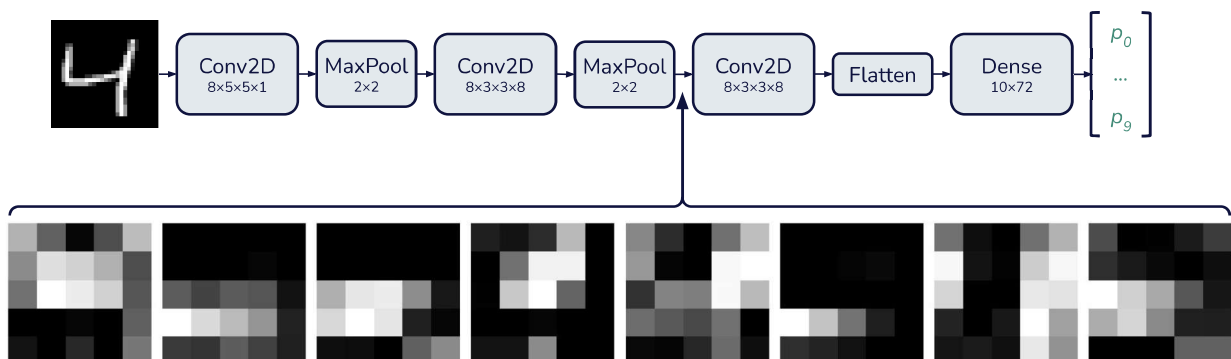
Convolutional Neural Network



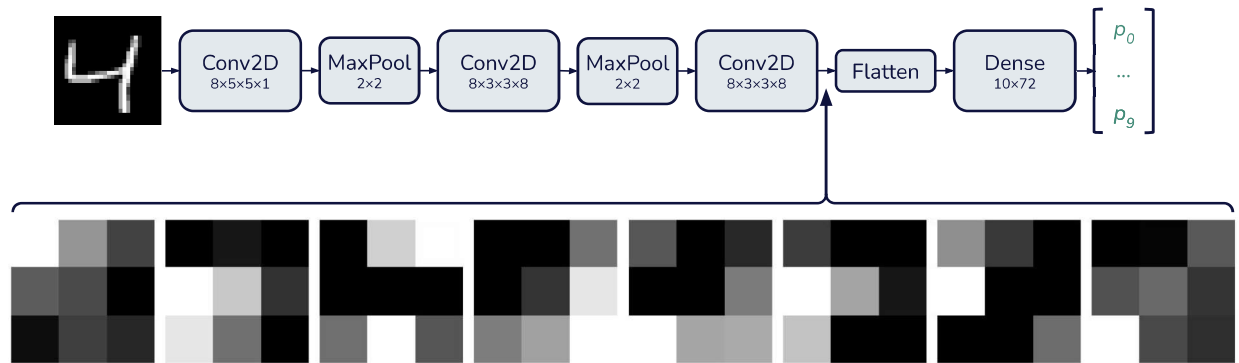
Convolutional Neural Network



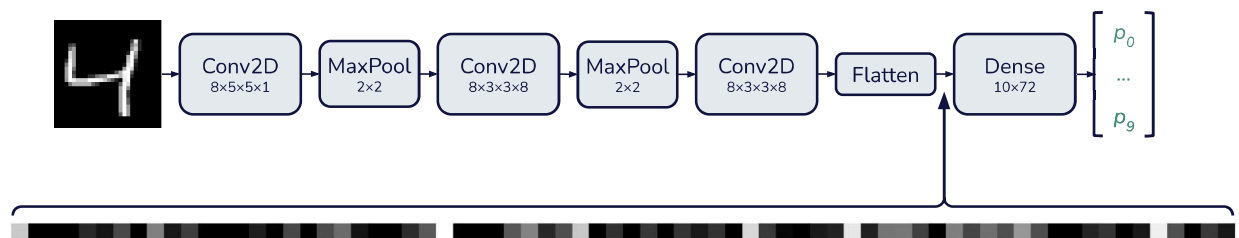
Convolutional Neural Network



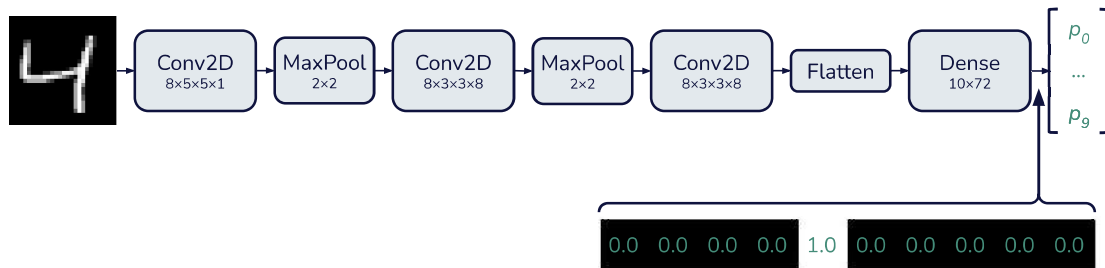
Convolutional Neural Network



Convolutional Neural Network



Convolutional Neural Network



Size/Flash Demands

- Relatively few parameters because they're reused for every activation
- Example (simple handwritten digit model):
 - Conv2D-1: $8 \times 5 \times 5 \times 1 + 8 = 208$ parameters
 - Conv2D-2: $8 \times 3 \times 3 \times 8 + 8 = 584$ parameters
 - Conv2D-3: $8 \times 3 \times 3 \times 8 + 8 = 584$ parameters
 - Dense-1: $10 \times 72 + 10 = 730$ parameters
 - Total: 2,106 parameters

Compute Demands

- Compute intensive due to repeated convolutions over images
- Example (simple handwritten digit model):
 - Conv2D-1: $24 \times 24 \times 8 \times 5 \times 5 \times 1 = 115,200$ multiplications
 - Conv2D-2: $10 \times 10 \times 8 \times 3 \times 3 \times 8 = 57,600$ multiplications
 - Conv2D-3: $3 \times 3 \times 8 \times 3 \times 3 \times 8 = 5,184$ multiplications
 - Dense-1: $10 \times 72 = 720$ multiplications
 - Total: 178,704 multiplications

Activation size

Filter size

Memory (RAM) Demands

- Memory demanding due to 3D activations
- Example (simple handwritten digit model):
 - Conv2D-1: $24 \times 24 \times 8 = 4,608$ floats
 - MaxPool-1: $12 \times 12 \times 8 = 1,152$ floats
 - Conv2D-2: $10 \times 10 \times 8 = 800$ floats
 - MaxPool-2: $5 \times 5 \times 8 = 200$ floats
 - Conv2D-3: $3 \times 3 \times 8 = 72$ floats
 - Dense-1: 10 floats
 - Peak demand: $4,608 + 1,152 = 5,760$ floats

Convolutional Architectures

How modern computer vision models work

The ImageNet Challenge

- Introduced 2010 with 21,841 classes
- Simplified 2012 with 1,000 classes
 - Training set: 1,281,167 images
 - Validation set: 50,000 images
 - Test set: 100,000 images
- CNN architectures compared by accuracy on test set

The ImageNet Challenge

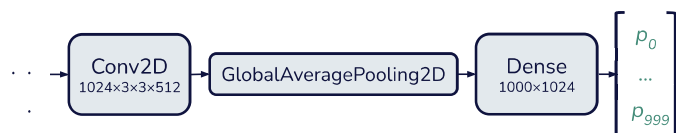
Model Architecture	Year	Layers	Parameters	Accuracy
AlexNet	2012	8	61.6M	57%
GoogLeNet	2014	22	6.6M	70%
ResNet	2015	152	60.4M	77%
Xception	2016	81	22.9M	79%
EfficientNet B7	2019	438	66.7M	84%
MobileNet V2	2018	105	3.5M	72%
MobileNet V3 Large	2019	Unknown	5.4M	76%
EfficientNet B0	2019	132	5.3M	77%

GoogLeNet: Global Average Pooling

- **Global Average Pooling** computes the average of each feature map:

$$y_k = \frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w x_{i,j,k}$$

- Used as replacement for Flatten before final classification layer:

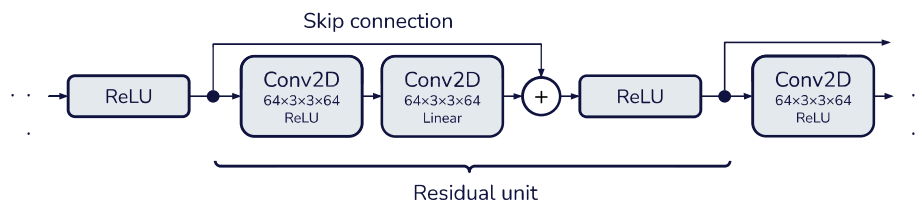


ResNet: Skip Connections

- **Skip connections** adds the output of layer $l-2$ to the pre-activation of layer l :

$$\mathbf{y}^{(l)} = \phi(\mathbf{y}^{(l-2)} + \mathbf{z}^{(l)})$$

- Prevents vanishing gradients $\Rightarrow$ Allows much deeper models



Xception: Depthwise Separable Convolution

A **Depthwise Separable Convolution** layer separates full convolution into depthwise and pointwise convolution:

$$a_{i,j,c} = \phi \left(b_c + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} W_{u,v,c} \cdot x_{i+u,j+v,c} \right)$$
$$y_{i,j,k} = \phi \left(b'_k + \sum_{c=0}^{C-1} W'_{k,c} \cdot a_{i,j,c} \right)$$



Depthwise Separable Convolution

- Reduces weights and multiplications by factor $(H \times W + K) / H \times W \times K$
- Example:
 - $h \times w$ (activation size) = 96×96
 - C (input channels) = 32
 - K (output channels) = 64
 - $H \times W$ (filter size) = 3×3
 - Weights:
 - Regular 2D-convolution: $K \times H \times W \times C = 18,432$ weights
 - DW separable convolution: $(H \times W + K) \times C = 2,336$ weights
 - Multiplications:
 - Regular 2D-convolution: $h \times w \times K \times H \times W \times C = 169,869,312$ multiplications
 - DW separable convolution: $h \times w \times (H \times W + K) \times C = 21,528,576$ multiplications

Sequence Modelling

How to process audio and sequences

Sequence Models: Example Applications

- Audio:
 - Keyword spotting
 - Speech recognition
- Sensor data:
 - Human activity recognition
 - Fall detection (Hövdning)
 - Predictive maintenance
- Natural language (text):
 - Email spam detection
 - Chatbot (e.g. ChatGPT)
- Video:
 - Gesture recognition

Sequence Models

- Input: Sequence of elements (often one vector per time step)
 - Audio: Spectrum vectors
 - Sensor data: Sensor samples as vectors
 - Natural language: Token embeddings
 - Video: Images
- Output:
 - **Sequence classification:** Class probabilities
 - Examples: Keyword spotter, Fall detection
 - **Sequence-to-sequence:** Another sequence
 - Example: Audio enhancement, Human activity over time, Translated sentence

Architectures for Sequence Modelling

- Recurrent Neural Networks (RNN)
- 1D-Convolution
- Transformers

Recurrent Neural Networks (RNN)

- Goes over the input sequence $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ from start to end
- For every step t from 1 to T :

$$\mathbf{h}_t = \phi(W\mathbf{x} + U\mathbf{h}_{t-1} + \mathbf{b})$$

- $\mathbf{h}_t$ is the **hidden state** at step t
- Output typically $\mathbf{y} = \mathbf{h}_T$
- Variants:
 - Long Short-Term Memory (LSTM)
 - Gated Recurrent Unit (GRU)
- Limited support in microcontroller inference libraries



1D Convolution

- Same as 2D-convolution without the height dimension
- Slides filters over the length of the sequence

1D Convolution Layer

The **1D convolution layer** with valid padding and stride 1:

$$a_{j,k} = \phi \left(b_k + \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} W_{k,v,c} \cdot x_{j+v,c} \right)$$

$\mathbf{x} \in R^{T \times C}$ is the feature matrix,

$\mathbf{W} \in R^{K \times W \times C}$ are the filter weights,

$\mathbf{b} \in R^K$ are the filter biases,

$W, C \in Z^+$ are the filter dimensions,

$\phi(\cdot)$ is the activation function,

$\mathbf{a} \in R^{(T-W+1) \times K}$ is the activation,

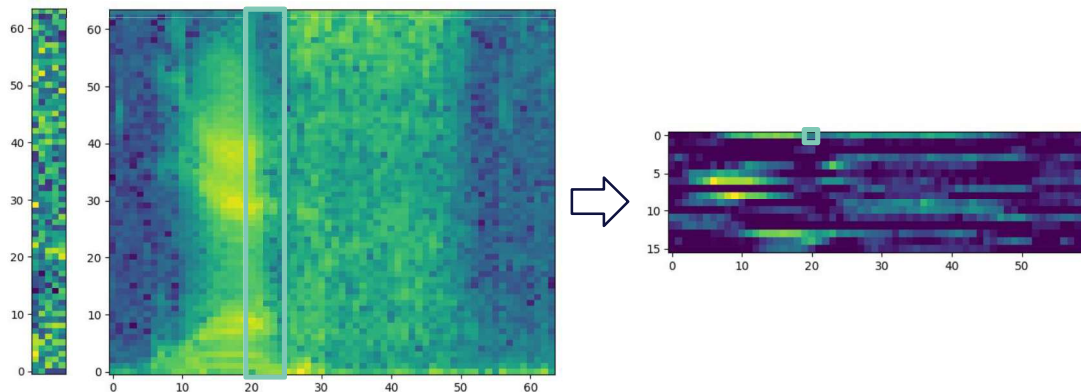
K is the number of filters.

1D Convolution Layer

- Multiple filters, each with shape (*width* $\times$ *channels*)
- Each filter produces a new output channel
- Properties:
 - Number of filters, e.g. 16
 - Filter width, e.g. 5
 - *channels* defined by input sequence
 - Activation
 - Padding: valid or same
 - Stride, by default 1

1D Convolution: Example

- 1D convolution with 16 filter of width 5
 - Input size: 64x64
 - Resulting activation size: 60x15



1D Convolution Layer

- Layer weights $\mathbf{W}$ are 3D matrices
 - *number of filters $\times$ width $\times$ number of input channels*
- Layer biases $\mathbf{b}$ are vectors (1D)
 - *number of filters*
- Inputs $\mathbf{x}$ and activations $\mathbf{a}$ are 2D matrices
 - *width $\times$ channels*

Transformers

- Uses the **attention** mechanism
- Mostly used for language models, but applicable to any sequence problem
- **Vision transformers** (ViT) for images
- Currently no support in microcontroller inference libraries

